Grouper: Optimal Assignment Workflows for Higher Education

by Mingyuan Zhang, Kevin Lam, and Vik Gopal

Abstract Grouper is an R package for formulating and solving allocation problems that arise in higher education. The package provides three mixed-integer optimization workflows. They cover diversity-based assignment of students to groups, preference-based assignment of students to topics, and multi-role workload allocation under fairness, preference, and priority constraints. Existing optimization tools in R provide powerful low-level modeling and solver interfaces, but instructors and administrators often need formulations that connect institutional rules to interpretable allocation outputs. Grouper addresses this gap through policy-aware mathematical models, structured input formats, configurable constraints and objective weights. It offers consistent entry and exit points to and from the model. Users can adapt group-size limits, topic capacities, preference weights, workload caps, fairness terms, and priority rules without rewriting the underlying model code. This paper presents the models, package design, implementation, and a detailed workflow demonstration, illustrating how Grouper can support reproducible and transparent allocation decisions in higher-education institutions.

1 Introduction

Allocation problems are a recurring feature of higher education. Instructors and administrators routinely need to assign students to groups, topics, supervisors, teaching duties, or other limited resources while balancing preferences, capacity, fairness, and institutional rules. These decisions often involve several competing objectives. For example, a course may need diverse project groups, a program may need to match self-formed groups to topic slots, and a department may need to distribute teaching or support duties fairly across staff.

In many cases, the allocation policy is clear in principle but difficult to operationalize. Manual allocation can be time-consuming, difficult to reproduce, and hard to audit when multiple constraints and policy goals interact.

The **grouper** package provides a high-level R workflow for optimization-based allocation in higher education. It currently supports three policy-aware workflows. These address diversity-based assignment of students to groups, preference-based assignment of students to topics, and multi-role workload allocation.

The contribution of **grouper** is both mathematical and computational. Mathematically, it encodes recurring allocation priorities such as diversity, preference satisfaction, capacity control, workload fairness, and priority-aware assignment as explicit objectives and constraints. Computationally, it exposes these models through a common R workflow in which users prepare structured data, specify policy parameters, solve the model through existing R optimization infrastructure, and inspect allocation results. This design allows users to adapt allocation rules to different courses, cohorts, departments, or workload settings without rewriting the underlying model code, making the allocation process more reproducible and transparent than ad hoc manual assignment.

This paper introduces the mathematical models, implementation, and main workflows of **grouper**. The next section reviews related work, including general-purpose R optimization tools and existing higher-education allocation approaches. In Section 2.2, we then present the three allocation models, and then describe how the package implements them. In Sections 2.4 and 2.5, we demonstrate the use of the package on small easy-to-understand examples and on real-life ones. Finally, the paper concludes with a discussion in Section 2.6.

Related work

Optimization-based allocation has a long history in higher-education planning. Student-project allocation, group formation, and workload distribution can often be expressed as mathematical programming problems in which assignment decisions are represented by binary or integer variables and institutional rules are encoded as constraints. Prior work has used integer programming approaches to assign students to projects or topics while accounting for preferences, capacities, and other institutional constraints (Anwar and Bahaj, 2003; Meyer, 2009; Ramotsisi et al., 2022).

In R, users can access a rich ecosystem of general-purpose optimization tools. Packages such as `ompr` provide algebraic modeling interfaces for mixed-integer programming (Schumacher, 2025a),

while `ROI` provides a common interface to multiple solver plugins (Hornik et al., 2024; Schumacher, 2025b).

Solver backends such as `Gurobi`, `GLPK`, and `HiGHS` can then be used to solve the resulting optimization problems (Gurobi Optimization, LLC, 2026; Free Software Foundation, 2026; HiGHS Developers, 2026). These tools are powerful and flexible, but they operate at a relatively low level of abstraction. Users must still define the decision variables, constraints, objective functions, data transformations, and post-processing steps for each allocation problem.

Several tools are closer to higher-education assignment. Student-project allocation has been studied through integer programming, and web-based or standalone implementations exist for specific versions of the problem (Anwar and Bahaj, 2003; Meyer, 2009). On CRAN, **golfr** provides classroom group assignment functionality with a focus on minimizing repeated pairings across multiple rounds (Kim and Nolte, 2025). These tools address important sub-problems, but they are generally designed around a single assignment setting or algorithmic goal.

grouper differs from these approaches by providing a higher-level package interface for multiple higher-education allocation workflows. The package does not replace lower-level optimization software; instead, it packages recurring models into documented workflows that combine structured input preparation, configurable policy parameters, mixed-integer model construction, solver execution, and interpretable output generation. To our knowledge, no existing R package provides a comparable orchestration layer for diversity-based student grouping, preference-based topic allocation, and multi-role workload allocation within one extensible framework.

2 Mathematical allocation models

Each **grouper** workflow is formulated as an integer or mixed-integer program. The objective describes the allocation goal, while the constraints define which allocations are feasible.

Diversity-based assignment

Purpose and inputs

Consider N students arranged into G self-formed groups. Each group is assigned to one of T topics, and topic t may have between r_{min} and r_{max} repetitions. The model is applicable to tutorial sections, lab teams, interdisciplinary discussion groups, and capstone teams where instructors wish to form heterogeneous final groups. Take note that, although the model is formulated with great flexibility by including repetitions of topics, in practice, this model typically runs with $G = N$ (i.e. groups of individuals), and $r_{max} = r_{min} = 1$ (i.e. T treated as the number of groupings to be created).

The user supplies the student data, the self-formed-group column, demographic information or a dissimilarity matrix, and optionally a numeric skill measure. The extractor constructs m_{ig}, which equals one when student i belongs to self-formed group g, the pairwise dissimilarity values d_{ij}, and the optional skill values s_i.

Objective

$$\max \quad w_1 \left(\sum_{i=1}^{N-1} \sum_{j=i+1}^{N} \sum_{t=1}^{T} \sum_{r=1}^{r_{max}} z_{ijtr} d_{ij} \right) + w_2 (s_{\min} - s_{\max}).$$

Here z_{ijtr} indicates whether students i and j are assigned to the same repetition r of topic t, while s_{min} and s_{max} correspond to the maximum and minimum total skill levels across all assigned groups. The first term in the objective function maximizes pairwise dissimilarity within the final groups. The second minimizes the difference between their maximum and minimum total skill. The non-negative weights w_1 and w_2 determine the relative priority of the two terms and are normally chosen to sum to one. If skill is not supplied, the second term is omitted.

Key constraints

Let

$$x_{gtr} = \begin{cases} 1, & \text{if group } g \text{ is assigned to repetition } r \text{ of topic } t, \\ 0, & \text{otherwise.} \end{cases}$$

Each self-formed group must be assigned exactly once:

$$\sum_{t=1}^{T}\sum_{r=1}^{r_{max}} x_{gtr} = 1, \quad \forall g. \tag{1}$$

This equality preserves every self-formed group as a unit. It rules out both leaving a group unassigned and assigning the same group to multiple topic-repetition combinations.

The binary variable z_{ijtr} records whether students i and j are both assigned to the topic-repetition combination:

$$\begin{aligned} z_{ijtr} &\leq \sum_{g=1}^{G} m_{ig} x_{gtr}, \\ z_{ijtr} &\leq \sum_{g=1}^{G} m_{jg} x_{gtr}, \qquad \forall i < j, t, r. \\ z_{ijtr} &\geq \sum_{g=1}^{G} m_{ig} x_{gtr} + \sum_{g=1}^{G} m_{jg} x_{gtr} - 1, \end{aligned} \tag{2}$$

The first two inequalities prevent z_{ijtr} from being one unless both students are in the slot. The third forces it to one when both are present. Consequently, d_{ij} contributes to the objective exactly when the student pair appears in the same final group.

Let $a_{tr} = 1$ when repetition r of topic t is active. The following constraints link activation to assignment and bound the number of repetitions:

$$\begin{aligned} a_{tr} &\geq x_{gtr}, && \forall g, t, r, \\ a_{tr} &\leq \sum_{g=1}^{G} x_{gtr}, && \forall t, r, \\ \sum_{r=1}^{R_t} a_{tr} &\geq r_{\min}, && \forall t, \\ \sum_{r=1}^{R_t} a_{tr} &\leq r_{\max}, && \forall t. \end{aligned} \tag{3}$$

The first two conditions make a_{tr} an indicator for whether the slot is used. The final two then count the used repetitions of each topic and keep that count between the instructor-specified limits $r_{\min}$ and $r_{\max}$.

The number of students in each active topic-repetition combination is bounded:

$$\begin{aligned} \sum_{i=1}^{N}\sum_{g=1}^{G} m_{ig} x_{gtr} &\geq a_{tr} n_{min}, \\ \sum_{i=1}^{N}\sum_{g=1}^{G} m_{ig} x_{gtr} &\leq a_{tr} n_{max}, \end{aligned} \qquad \forall t, r. \tag{4}$$

Multiplication by a_{tr} switches the bounds off for an inactive slot. For an active slot, the total number of students contributed by its assigned self-formed groups must lie between n_{min} and n_{max}, which correspond to the number of students each final group should contain.

When skill balancing is used, $s_{\min}$ and $s_{\max}$ bound the total skill assigned to each active topic-repetition combination. Let $M = \sum_{i=1}^{N} s_i$:

$$\begin{aligned} \sum_{i=1}^{N}\sum_{g=1}^{G} m_{ig} x_{gtr} s_i + M(1 - a_{tr}) &\geq s_{\min}, \\ \sum_{i=1}^{N}\sum_{g=1}^{G} m_{ig} x_{gtr} s_i &\leq s_{\max}, \end{aligned} \qquad \forall t, r. \tag{5}$$

These bounds make $s_{\min}$ and $s_{\max}$ the lowest and highest total skill among active slots. Maximizing $s_{\min} - s_{\max}$ therefore minimizes the skill spread. Inactive candidate repetitions are excluded from this comparison because $M(1 - a_{tr})$ relaxes the lower bound when $a_{tr} = 0$.

For easy reference, Table 1 lists the input parameters for this model, along with explanations of

how they impact the final group assignments.

Table 1: Parameter effect guide for the diversity-based assignment model (directional, holding other settings fixed). Parameter names follow how they appear as arguments in code.

Parameter	If increased	Trade-off
w1	Places greater relative emphasis on pairwise diversity	May place less emphasis on skill balance
w2	Places greater relative emphasis on reducing skill differences	May accept lower pairwise diversity
nmin	Requires more students in each active topic-repetition combination	Tightens feasibility
nmax	Permits more students in each topic-repetition combination	Increases flexibility but may permit less even group sizes
rmin	Requires more active repetitions of every topic	Tightens feasibility and spreads students across more repetitions
rmax	Permits more active repetitions of every topic	Increases flexibility but may fragment the cohort
n_topics	Requires assignments across more topics when 'rmin' is positive	Increases model size and may tighten feasibility

Preference-based assignment

Purpose and inputs

Unlike the diversity-based model, which assigns groups according to student attributes and the desired composition of the final slots, this model is used when groups have expressed preferences over the available topics. Just as before, the self-formed groups remain intact, and the main decision is where each group should be placed rather than which student profiles should be brought together.

Consider N students arranged into G self-formed groups. Each group is assigned to a topic t from T base topics, where T lies between r_{min} and r_{max}. A topic may contain B subgroups, giving BT topic-subgroup combinations. This setting arises in the allocation of pre-determined project topics, case studies, supervisors, lab rotations, presentation themes, and tutorial or project streams. The concept of sub-topics is specific to our use-case (see Section 2.5.2, but it is not necessary. By setting $B = 1$, it is possible to use this **grouper** model in a more traditional setting.

The user supplies the student data, self-formed-group column, and preference matrix. The extractor obtains m_{ig}, (self-formed) group sizes n_g, and preference scores p_{tg}, where higher values indicate stronger preferences.

Objective

$$\max \sum_{g=1}^{G} \sum_{t=1}^{BT} \sum_{r=1}^{r_{max}} x_{gtr} n_g p_{tg}.$$

Here $x_{gtr} = 1$ when group g is assigned to repetition r of topic-subgroup combination t. Multiplication by n_g means that an assignment affecting a larger group contributes more to total student-level preference satisfaction.

Key constraints

Every self-formed group must be assigned exactly once:

$$\sum_{t=1}^{BT} \sum_{r=1}^{r_{max}} x_{gtr} = 1, \quad \forall g. \tag{6}$$

As in the diversity model, this equality keeps each self-formed group intact and assigns it to one, and only one, topic-subgroup repetition.

Let $a_{tr} = 1$ when repetition r of topic-subgroup combination t is active. Activation is linked to the assignments by

$$\begin{aligned} a_{tr} &\geq x_{gtr}, && \forall g, t, r, \\ a_{tr} &\leq \sum_{g=1}^{G} x_{gtr}, && \forall t, r. \end{aligned} \tag{7}$$

These conditions ensure that a repetition is marked active exactly when at least one self-formed group is assigned to it. This indicator is then used in the repetition and capacity constraints below.

The number of repetitions of each base topic is bounded by

$$\begin{aligned} \sum_{r=1}^{R_t} a_{tr} &\geq r_{\min}, \\ \sum_{r=1}^{R_t} a_{tr} &\leq r_{\max}, \end{aligned} \qquad \forall t \in \{1, \ldots, T\}. \tag{8}$$

The constraints are written for the first subgroup of each base topic. They require every offered topic to have between $r_{\min}$ and $r_{\max}$ active repetitions.

The number of active repetitions is kept equal across subgroups belonging to the same base topic:

$$\sum_{r=1}^{R_t} a_{tr} = \sum_{r=1}^{R_t} a_{(bT+t)r}, \quad \forall t \in \{1, \ldots, T\}, \quad b = 1, \ldots, B-1. \tag{9}$$

With topic-subgroup columns ordered as $T_1S_1, \ldots, T_TS_1, T_1S_2, \ldots$, this equality gives every subgroup of a base topic the same number of active repetitions. The limits in (8) therefore propagate to all B subgroups.

Every active topic-subgroup repetition must satisfy its student-capacity bounds:

$$\begin{aligned} \sum_{i=1}^{N} \sum_{g=1}^{G} m_{ig} x_{gtr} &\geq a_{tr} n_{min}, \\ \sum_{i=1}^{N} \sum_{g=1}^{G} m_{ig} x_{gtr} &\leq a_{tr} n_{max}, \end{aligned} \qquad \forall t \in \{1, \ldots, BT\}, r. \tag{10}$$

The membership matrix converts group-level assignments into student counts. Thus, every active repetition must contain at least n_{min} and at most n_{max} students, while inactive repetitions remain empty. Table 2 summarises the inputs for the preference-based assignment model.

Table 2: Parameter effect guide for the preference-based assignment model (directional, holding other settings fixed).

Parameter	If increased	Trade-off
n_topics	Provides more base topics, each subject to the repetition rules	Increases model size and may tighten feasibility
B	Adds more subgroups to every topic	Permits finer organization but adds balancing requirements
nmax	Permits more students in each repetition	Increases flexibility but may permit crowded groups
rmin	Requires more repetitions of every topic and corresponding subgroup	Tightens feasibility and may reduce achievable preference scores
rmax	Permits more repetitions	Increases flexibility but may fragment allocations

Multi-role workload allocation

Purpose and inputs

The third model addresses a general multi-role workload allocation problem that arises in higher-education departments. Individuals are assigned discrete work units across three roles:

1. teaching assistantship (TA)
2. grading assignments and exams (GR)

3. lighter extra duties such as invigilating exams, proctoring, and other administrative support work (E)

The allocation satisfies task demand, individual capacity limits, and institutional policy constraints. The extraction step receives three order-aligned input objects:

1. an individual table,
2. optional role-specific preference matrices, and
3. a role-demand matrix.

The individual table supplies cohort year and previous TA and GR workload, which are converted into the historical workload vectors $t_i^{(1)}$ and $g_i^{(1)}$. Row i in each preference matrix must correspond to row i in the individual table, and demand row j must correspond to preference column j. The preference matrices supply P_{ij}^{TA} and P_{ij}^{GR}. The demand matrix supplies d_{jr}, the required number of units for task j and role r. A user-configurable score s_i is used to guide the allocation of E units. E demand can either be supplied directly or derived from total semester capacity after submitted TA and GR demand have been accounted for.

Objective

Let X_{ijr} be the non-negative integer units of role r assigned to individual i for task j, where $r \in \{\mathrm{TA}, \mathrm{GR}, \mathrm{E}\}$. Define annual TA and GR workload as

$$T_i = t_i^{(1)} + \sum_j X_{ij,\mathrm{TA}}, \qquad G_i = g_i^{(1)} + \sum_j X_{ij,\mathrm{GR}}$$

Let Y_{TA} and Y_{GR} denote the cohorts protected for TA and GR, respectively. The two cohorts may be the same or different. The general objective is

$$\begin{aligned} \min \quad & \alpha_{\mathrm{TA}}(T_{\max} - T_{\min}) + \alpha_{\mathrm{GR}}(G_{\max} - G_{\min}) \\ & - \beta_{\mathrm{TA}} \sum_{i,j} P_{ij}^{\mathrm{TA}} X_{ij,\mathrm{TA}} - \beta_{\mathrm{GR}} \sum_{i,j} P_{ij}^{\mathrm{GR}} X_{ij,\mathrm{GR}} \\ & - \phi \sum_{i,j} s_i X_{ij,\mathrm{E}} + \rho_{\mathrm{TA}} \sum_{i \in Y_{\mathrm{TA}}} w_i^{\mathrm{TA}} + \rho_{\mathrm{GR}} \sum_{i \in Y_{\mathrm{GR}}} w_i^{\mathrm{GR}}. \end{aligned}$$

The first two terms penalize the annual workload spread for TA and GR roles. The next two terms reward role-specific preference fit, while the fifth term rewards assigning E units according to the supplied priority score. The final two terms penalize violations of the protected-cohort soft limits for TA and GR.

Each objective component is optional. By setting the corresponding hyperparameter to 0 or NULL, users can remove that component from the model during construction. This allows the same formulation to represent policies that use only a subset of the available criteria, without requiring users to rewrite the underlying optimization model.

Key constraints

Every task-role demand must be met exactly:

$$\sum_i X_{ijr} = d_{jr}, \quad \forall j, r. \tag{11}$$

Each individual's annual workload is fixed at $2C$:

$$T_i + G_i + \sum_j X_{ij,\mathrm{E}} = 2C, \quad \forall i. \tag{12}$$

The role-specific fairness variables bound annual workload outside the corresponding protected cohort:

$$\begin{aligned} T_{\min} \le T_i \le T_{\max}, \quad \forall i \notin Y_{\mathrm{TA}}, \\ G_{\min} \le G_i \le G_{\max}, \quad \forall i \notin Y_{\mathrm{GR}}. \end{aligned} \tag{13}$$

For protected individuals, separate slack variables permit but penalize current-semester workload above the corresponding soft threshold:

$$\begin{aligned} \sum_j X_{ij,\mathrm{TA}} &\le t_{\max}^{(P)} + w_i^{\mathrm{TA}}, \quad \forall i \in Y_{\mathrm{TA}}, \\ \sum_j X_{ij,\mathrm{GR}} &\le g_{\max}^{(P)} + w_i^{\mathrm{GR}}, \quad \forall i \in Y_{\mathrm{GR}}. \end{aligned} \tag{14}$$

Optional role-wide lower and upper bounds are imposed only when supplied:

$$\ell_r \le \sum_j X_{ijr} \le u_r, \quad \forall i, r. \tag{15}$$

Historical workload allows the model to balance past and current-semester duties together. When previous workload is unavailable, `single_semester = TRUE` generates uniform synthetic history with $t_i^{(1)} = 0$ and $g_i^{(1)} = C$. This leaves C units per individual for the current allocation without changing GR workload spread. Please refer to Table 3 for a summary of inputs.

Table 3: Parameter effect guide for the multi-role workload allocation model (directional, holding other settings fixed).

Parameter	If increased	Trade-off
'alpha_ta', 'alpha_gr'	Increases the penalty on annual workload spread for the corresponding role	Can improve role-specific fairness at the expense of preference, priority, or protection terms
'beta_ta', 'beta_gr'	Increases the reward for the corresponding role's preference scores	Can improve role-specific preference attainment at the expense of other objective terms
'phi'	Increases the reward for assigning E to individuals with larger scores	Can improve priority alignment at the expense of the other objective terms
'rho_ta', 'rho_gr'	Makes the corresponding protected-cohort slack more expensive	Discourages overload but does not alter feasibility because slack remains available
'ta_protected_max', 'gr_protected_max'	Raises the corresponding protected cohort's soft threshold	Weakens protection and can reduce the slack penalty; formal feasibility is unchanged
'C'	Raises every individual's required annual workload from 2C	Requires two additional workload units per person for each unit increase and may cause infeasibility if aggregate demand is incompatible
'ta_min', 'gr_min', 'e_min'	Raises the required minimum for the corresponding role	Tightens feasibility and forces more of that role onto every individual
'ta_max', 'gr_max', 'e_max'	Raises the permitted maximum for the corresponding role	Weakly enlarges the feasible set and may allow greater concentration

3 The grouper package

Figure 1 summarizes the architecture of **grouper** as a domain-specific implementation layer for the allocation models above. The pipeline accepts student information together with allocation attributes such as preferences, demand, capacities, and policy constraints. Inside the package, these inputs pass through data processing, model building, and solving. The final stage returns a raw solver result for diagnostics and reproducibility together with an allocation table for operational use.

Wrapper contract

Each stage in Figure 1 is exposed through a high-level wrapper, with model-specific functions available underneath for advanced use. This contract is intentionally stable across the supported assignment settings. Users can run a concise end-to-end pipeline for routine use, while retaining granular control over input preparation, model construction, solver choice, and post-processing when a local policy requires it.

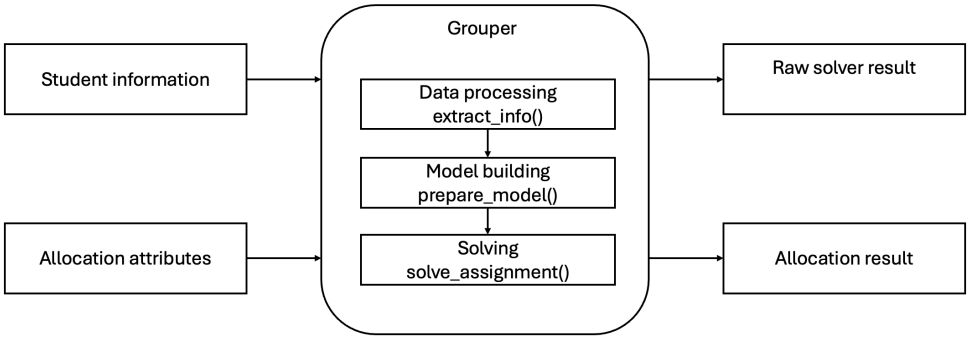

Figure 1: High-level package architecture.

In this section, we use the `assignment = "multirole"` argument to show the multi-role workload workflow. For example applications on the other models, the reader is referred to Sections 2.5 and 2.4.

`extract_info()`

The `extract_info()` function standardizes raw inputs into a list structure used by the model builders. For `assignment = "diversity"`, it accepts the student or group data frame, the self-formed-group column, and either a precomputed dissimilarity matrix or demographic columns from which dissimilarity is computed. For `assignment = "preference"`, it accepts the same group structure together with the group-topic preference matrix. For the multi-role workflow, `assignment = "multirole"` accepts the individual table, role-demand matrix, optional role-specific preference matrices, semester capacity, E-demand mode, optional E-allocation scores, and history mode. In all cases, the purpose of `extract_info()` is to translate user-supplied records into the mathematical objects used in the optimization model.

The following illustrative call uses `S`, `P_TA`, `P_GR`, and `D` to denote the individual table, TA preference matrix, optional GR preference matrix, and role-demand matrix.

```
df <- extract_info(
  "multirole",
  student_df = S,
  d_mat = D,
  p_ta_mat = P_TA,
  p_gr_mat = P_GR
)
```

`prepare_model()`

The `prepare_model()` function converts standardized inputs and policy parameters into an `ompr` mixed-integer optimization model. For diversity-based assignment, it combines the extracted membership and dissimilarity information with topic, repetition, size, and objective-weight parameters. For preference-based assignment, it builds the topic-slot assignment model from group sizes, preference scores, and slot-capacity parameters. For multi-role workload allocation, it passes the extracted workload inputs and policy weights to the role-allocation model. The returned object is a solver-ready model rather than an allocation table, which lets users inspect or modify the model before solving when needed.

```
m <- prepare_model(
  df,
  assignment = "multirole",
  alpha_ta = a_ta,
  alpha_gr = a_gr,
  beta_ta = b_ta,
  beta_gr = b_gr,
  phi = p,
  rho_ta = r_ta,
  rho_gr = r_gr
)
```

solve_assignment()

The solve_assignment() function solves a prepared model and converts the raw solver solution into an allocation output. The function uses ompr.roi to call an ROI-backed solver, with solver = "glpk", "highs", or "gurobi" selecting the backend when the corresponding plugin or solver is available. It returns a list with model_result, the raw output from ompr::solve_model() for diagnostics and reproducibility, and output, the post-processed assignment table. For the student-group workflows, the output maps groups back to selected topics or slots; for the multi-role workload workflow, the output is a wide individual-by-task-role allocation table. For multi-role outputs, the course-code vector J is passed as course_codes, and its order must match the row order of the demand matrix used during extraction.

```
out <- solve_assignment(
  model = m,
  assignment = "multirole",
  solver = "glpk",
  student_df = S,
  course_codes = J
)
```

Conceptually, all supported workflows share the same pipeline contract. Institutional data are converted into standardized model inputs and policy choices are translated into constraints and objective terms. Solver results are converted into interpretable allocation outputs. This contract is the main software contribution of **grouper** relative to lower-level modeling toolkits such as ompr, ROI, and solver interfaces. Those tools provide flexible optimization infrastructure, whereas **grouper** provides a domain-specific layer that connects higher-education data, institutional policy, model construction, and decision-ready allocation outputs.

4 Constructed Datasets

In order to understand how **grouper** works, we demonstrate its use on two small and artificially constructed datasets in this section. This allows new users to understand the arguments and workflow better. Additional constructed examples can be found in the package vignettes.

Diversity-based assignment

The dataset for this example dba_gc_ex001 is available with the package. The scenario here is that there are four students (hence four rows), each in their own individual group. The goal is to divide the four students into two groups, with each group having maximal diversity, as measured by their majors. It should be clear from inspection of the data that one group should comprise students 1 and 3, and the other should comprise students 2 and 4. While the id column is strictly not necessary for **grouper** to work, it is good to include it.

```
dba_gc_ex001
```

```
#>   id major skill groups
#> 1  1     A     1      1
#> 2  2     A     1      2
#> 3  3     B     3      3
#> 4  4     B     3      4
```

The next three lines of code demonstrate how to apply the DBA model to the above dataset, using the functions described earlier in Section 2.3.

```
dba_ex1_list <- extract_info(assignment = "diversity", dframe = dba_gc_ex001,
                        demographic_cols = 2, skills = 3, self_formed_groups = 4)

dba_ex1_model <- prepare_model(dba_ex1_list, assignment = "diversity",
                               w1 = 1.0, w2 = 0.0,
                               n_topics = 2, nmin = 2, nmax = 2, rmin = 1, rmax = 1 )

dba_ex1_result <- solve_model(dba_ex1_model, with_ROI(solver = "glpk"))
```

In order to retrieve the results in a usable format, with the assigned groupings mapped to the original dataset, **grouper** provides a convenience function `assign_groups()` to perform the required table join.

```
dba_ex1_output_df <- assign_groups(model_result = dba_ex1_result,
                                   assignment = "diversity", dframe = dba_gc_ex001,
                                   group_names = "groups")
```

The final output dataset, displayed in Table 4, is identical to the original data `dba_gc_ex001` except for the columns `topic` and `rep`. The allocation agrees with the stated intuition that students 1 and 3, and students 2 and 4, should be assigned together.

Table 4: The assigned groupings for the constructed DBA dataset. Students 2 and 4 have been assigned to topic 1 and students 1 and 3 to topic 2.

topic	rep	group	id	major	skill
1	1	2	2	A	1
1	1	4	4	B	3
2	1	1	1	A	1
2	1	3	3	B	3

Preference-based assignment

The second example dataset, which also ships with the package, comprises 8 students. Each student is in a self-formed group of size 2, indicated via the `grouping` column. Here is a listing of the dataset:

```
pba_gc_ex002
```

```
#>   id grouping
#> 1  1        1
#> 2  2        1
#> 3  3        2
#> 4  4        2
#> 5  5        3
#> 6  6        3
#> 7  7        4
#> 8  8        4
```

Suppose that, for this set of students, the instructor wishes to assign students into two topics, with each topic having two sub-groups. This requires the preference matrix to have 4 rows (one for each self-formed group), and 4 columns (one for each topic-subgroup combination). Take note that the ordering of topics/subtopics must be:

Topic1-Subtopic1, Topic2-Subtopic1, Topic1-Subtopic2, Topic2-Subtopic2

The constructed preference matrix for this example is as follows:

```
pba_prefmat_ex002
```

```
#>      col1 col2 col3 col4
#> [1,]    4    3    2    1
#> [2,]    3    4    2    1
#> [3,]    1    2    4    3
#> [4,]    1    2    3    4
```

In this toy example, it is possible to assign each self-formed group to its optimal choice of topic-subtopic combination. In our solution, we should see that group 1 is assigned to subtopic 1 of topic 1, group 2 is assigned to sub-topic 1 of topic 2, and so on.

```
pba_ex2_list <- extract_info(assignment = "preference",
                             dframe = pba_gc_ex002, self_formed_groups = 2,
                             pref_mat = pba_prefmat_ex002 )

pba_ex2_model <- prepare_model(pba_ex2_list, assignment = "preference",
                               n_topics = 2, B = 2, nmin = 2, nmax = 2,
                               rmin = 1, rmax = 1 )

pba_ex2_result <- solve_model(pba_ex2_model, with_ROI(solver = "glpk"))

pba_ex2_output_df <- assign_groups(model_result = pba_ex2_result,
                                   assignment = "preference",
                                   dframe = pba_gc_ex002,
                                   params_list = list(n_topics = 2, B = 2),
                                   group_names = "grouping" )
```

As before, the purpose of the final line in the chunk is to merge the output with the original dataset. The allocation is shown in Table 5.

Table 5: The assigned groupings for the constructed PBA dataset. The number of rows in this dataset is equal to the number of students in the course.

topic	subtopic	rep	group	size	id
1	1	1	1	2	1
1	1	1	1	2	2
1	2	1	3	2	5
1	2	1	3	2	6
2	1	1	2	2	3
2	1	1	2	2	4
2	2	1	4	2	7
2	2	1	4	2	8

5 Examples from Past Semesters

This section contains real-life examples that motivated the models in **grouper**. In all three scenarios, we revisit semesters where allocation had been carried out manually, and consider how helpful it would have been, to have **grouper** available.

Diversity-based allocation

The final author of this paper teaches an interdisciplinary course together with a colleague from the English Language department. Students first learn about data science and linguistics in lectures. Then, in small groups, they discuss chosen topics from the viewpoint of linguistics and data science/statistics. As such, it is necessary to create groups with a good balance of Data Science and Statistics (dsds) majors, and non-dsds majors. In short, for this course, the instructors needed to divide tutorial classes of 25 - 30 students into 4 - 5 heterogeneous groups. Every semester that the course was taught, this procedure had to be carried out 5 - 6 times, since the total course size was typically 120 students. Figure 2 depicts the manual, and retrospective **grouper** output, for one such tutorial class of 25 students. The code to reproduce the analyses can be found in the supplementary material for this paper.

Preference-based allocation

The genesis of this particular model began in a project-based course, in which it was necessary to divide students into project teams with two sub-groups: one sub-group typically developed a user interface, and the other worked on data analysis. Each project team then needed to be assigned to one of a set of pre-determined project titles. As there were more titles than project teams, there was a need to assign multiple teams to the same title; hence the need for repetitions.

Table 6 shows how **grouper** could have benefited us if it had been used over the four semesters the course was taught.

Figure 2: Comparison between grouper allocation and manual allocation for diversity-based groupings in an interdisciplinary course. The top row corresponds to the outcomes if grouper had been available to carry out the allocation. The bottom row consists of the manual allocation outcomes. Notice that group 2 in the manual assignment was heavily imbalanced.

Consider the first semester we ran the course (row 1 in Table 6): the enrolment was 62 and there were 4 topic titles to be allocated. In the manual allocation carried out, the average preference of the self-formed groups was 8.16. *If we could have used* **grouper**, the average preference for this same group of students would have been 9.47 (higher is better).

Table 6: Breakdown of preference scores (per student). If grouper had been available, allocation could have been carried out in a more optimal manner, and much quicker!

Semester	Manual score	Grouper score	Class size	No. topics	No. project teams
1	8.16	9.47	62	4	7
2	13.10	13.30	108	7	13
3	12.70	13.50	89	7	11
4	9.48	9.66	151	5	18

Figure 3 confirms that in every semester, **grouper** could have improved the allocations. From the figure, it may not appear that the improvement was significant, but the reader must bear in mind that, given the class sizes, the manual allocation typically took 4 - 5 days effort.

Multi-role workload allocation in practice

This section evaluates the multi-role workload workflow using institutional planning data from AY2420, AY2510, and AY2520. The notation follows the department's academic-year convention, where AY denotes academic year, the first two digits indicate the academic year, and the final two digits indicate the semester. The three cases cover different cohort sizes, course offerings, and role-demand profiles, allowing us to assess the workflow across multiple operational settings and compare optimized schedules with the manual schedules used by the department. Student identifiers and course codes are anonymized throughout.

Table 7 summarizes the size and structure of each semester instance. Students and Courses describe the scale of the assignment problem, while TA, GR, and E report the required units for teaching assistantship, grading, and residual support, respectively. In this use case, TA is the preference-sensitive role, GR is primarily demand-driven, and E is assigned according to seniority. The columns Y1–Y4 show the cohort composition by year. AY2510 is the largest instance by both student and course count, while role demand and cohort structure vary across the three semesters.

The allocation uses the policy configuration from the original departmental planning exercise:

1. TA fairness
2. TA preference fit
3. seniority-guided E allocation

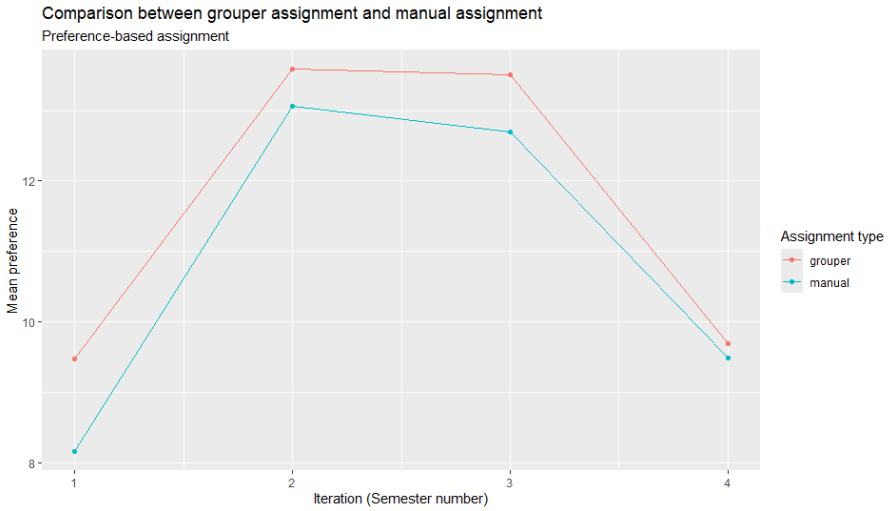

Figure 3: Comparison between grouper allocation and manual allocation for preference-based groupings. The red line corresponds to mean preference scores if grouper had been available to carry out the allocation. The green line corresponds to mean preference scores from the actual manual allocations. While the improvement may not appear to be substantial, one must take note that the manual allocation process could take 4 - 5 days, while grouper takes minutes.

Table 7: Scale, role demand, and cohort composition of the three multi-role workload datasets.

Semester	Students	Courses	TA	GR	E	Y1	Y2	Y3	Y4
AY2420	43	19	54	80	38	10	13	8	12
AY2510	48	25	50	104	38	15	12	11	10
AY2520	40	19	55	81	24	10	11	13	6

4. Year-1 TA protection

This is a subset of the generalized model. In particular, no GR preference term is evaluated because it was not part of the department's operational policy in this case.

The same wrapper workflow is applied to all three semesters. To avoid repeating nearly identical code, AY2520 is used only to demonstrate the input, model-building, and solving steps and to show one student-level workload distribution.

All three cases are formulated as two-semester workload problems. The current semester is optimized, while previous-semester TA and grading loads are supplied as `past_ta` and `past_gr`. This lets the model balance annual workload rather than treating each semester in isolation.

The workflow begins by standardizing the inputs. In this example, `student_df` records the anonymized student identifier, cohort year, and previous-semester TA/GR workload. The TA preference matrix P^{TA} is individual by course, and the demand matrix D is course by role. A GR preference matrix is not supplied for this policy configuration. The reproducibility setup fixes the student and course positions before matrix construction; `extract_info()` then uses those positions as supplied. Residual support demand is already included as the third role column, so `e_mode` is set to `"none"`.

```
df_multirole <- extract_info(
  assignment = "multirole",
  student_df = students_ay2520[, c("student_id", "year", "past_ta", "past_gr")],
  d_mat = D_ay2520,
  p_ta_mat = P_ay2520,
  e_mode = "none",
  C = 4
)
```

The resulting `df_multirole` object is the standardized input contract used by the model-building stage. It contains the TA preference matrix, role-demand matrix, semester capacity, seniority score, and past workload vectors in the order supplied to the extraction step.

The second step constructs the mixed-integer model. In this example, the policy weights are set to `alpha_ta = 2`, `beta_ta = 1`, `phi = 1`, and `rho_ta = 10`, encoding the trade-off between yearly TA

fairness, TA preference fit, seniority-guided residual support work, and Year-1 TA overload protection. The GR-specific weights are explicitly disabled. The bound `ta_protected_max = 1` limits Year-1 teaching assistantship load before slack is used, and `e_max = 1` keeps residual support duties from concentrating too heavily on any single student.

```
model_multirole <- prepare_model(
  df_list = df_multirole,
  assignment = "multirole",
  alpha_ta = 2,
  alpha_gr = NULL,
  beta_ta = 1,
  beta_gr = NULL,
  phi = 1,
  rho_ta = 10,
  rho_gr = NULL,
  protected_year_ta = 1,
  ta_protected_max = 1,
  e_max = 1
)
```

At this point, the optimization model is fully specified but not yet solved. The third step selects the solver and requests the operational output mapping. `GLPK` is used here because it is open-source and supports a reproducible example in a standard R environment.

```
multirole_solution <- solve_assignment(
  model = model_multirole,
  assignment = "multirole",
  solver = "glpk",
  student_df = students_ay2520,
  course_codes = demand_ay2520$course_code,
  name_col = "student_id",
  verbose = FALSE
)
```

The returned object contains `model_result`, which keeps the raw solver result for diagnostics, and `output`, which stores the allocation in a wide student-by-course-role table. Figure 4 plots the absolute objective value. A taller bar represents a more negative, and hence lower, objective value.

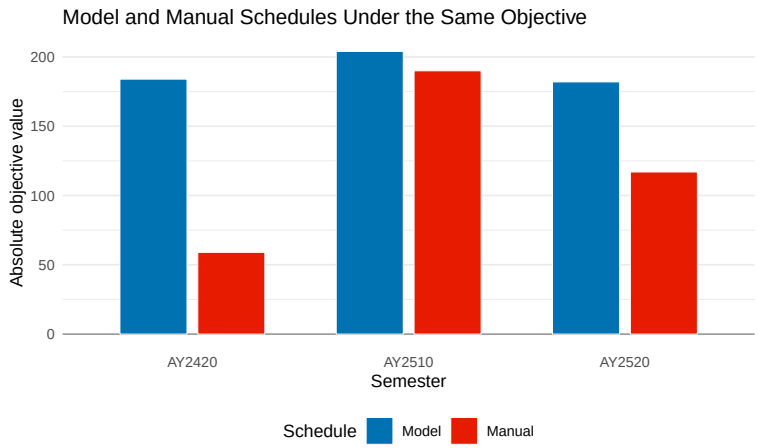

Figure 4: Model and manual schedules compared by absolute objective value across three semesters.

Both schedules are scored using the same set of hyperparameters and it can be observed that the model obtains a lower weighted objective in all three semesters, indicating that the optimized allocation better satisfies the policy criteria.

Figure 5 then shows how an optimized schedule is allocated at student level. For AY2520, it displays annual workload by student and cohort year. The lower part of each stacked bar is the optimized current-semester allocation, while the upper part is previous-semester workload carried into the annual balance.

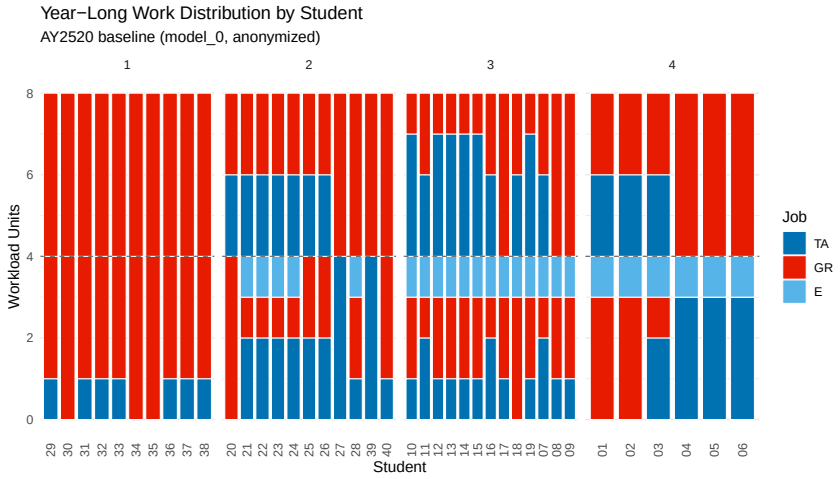

Figure 5: AY2520 model workload distribution by anonymized student and cohort year.

```
The dashed horizontal line marks the four-unit semester capacity. Faceting by
cohort year makes the Year-1 protection
and seniority-based residual-support pattern easier to inspect.

Year-1 students receive at most one current-semester TA unit, while residual-support units are
assigned only to students in Years 2 to 4. Among Years 2 to 4, annual TA workload
ranges from one to four units, giving the three-unit spread reported for the
baseline solution.

Figure \\ref{fig:fig-multi-role-objective-term-gap} examines the semester with
the largest model--manual objective gap, AY2420, by decomposing the gap into
weighted objective terms.


\begin{figure}[!htbp]

{\centering \includegraphics[width=0.78\linewidth,alt={Horizontal bar chart showing AY2420 manual-minus-model w

}

\caption{AY2420 weighted objective-term differences between manual and model schedules.}\label{fig:fig-multi-ro
\end{figure}

The decomposition makes the trade-offs explicit. In AY2420, the preference term
accounts for 125 objective units of the 125-unit total gap. The fairness term
adds 8 units to the manual objective, while the seniority-residual-support term
offsets the gap by 8 units and the Year-1 slack term is unchanged. In this
sense, \pkg{grouper} makes allocation rules explicit and searches the feasible
assignment space more systematically than an operational manual process.

### Hyperparameter effectiveness

A one-at-a-time AY2520 sensitivity check was used to examine whether local
changes to the objective weights alter the operational conclusions. In this
example, changing `alpha_ta` has the clearest effect on TA workload fairness
because it directly penalizes the annual TA spread. The
spread decreases from 5 units at `alpha_ta = 1` to 3 units at the manuscript
value `alpha_ta = 2`, and to 2 units in the returned solution at
`alpha_ta = 4`. Local changes to `beta_ta`, `phi`, and `rho_ta` have smaller
visible effects because the baseline solution already achieves high TA
preference attainment, assigns residual support toward senior students, and
uses no Year-1 TA-overload slack. The sensitivity check therefore supports
interpreting these active weights as local policy levers rather than universal
```

```
tuning constants.


The reproducible benchmark repeated the AY2520 solve 30 times for each
open-source solver. `GLPK` returned the same objective value,
\(-182\), in all 30 runs with a mean
runtime of 0.188 seconds and an observed
range from 0.179 to
0.238 seconds. `HiGHS` also returned
objective value \(-182\) in all 30
runs, with a mean runtime of 0.198
seconds and an observed range from 0.189
to 0.238 seconds. These timings indicate
that the example is small enough for either open-source backend to solve
interactively.

# Web frontend

\pkg{grouper} also ships with a \CRANpkg{shiny} [@R-shiny] frontend for users who prefer a
browser-based workflow over direct R scripting. The unified application is stored
under `inst/shiny/grouperWebApp/` and is built with \CRANpkg{bslib} [@R-bslib] and
interactive tables from \CRANpkg{DT} [@R-DT]. It is intended for instructors and
administrators who need to prepare allocation inputs, set policy parameters, run
an optimization model, inspect outputs, and export reusable assignment files
without writing the wrapper calls by hand. The landing page of the application
is shown in Figure \\ref{fig:fig-grouper-webapp-home}.

The following command launches the application locally.


``` r
library(shiny)
runApp(system.file(file.path("shiny", "grouperWebApp", "app.R"),
                   package = "grouper"))
```
```

The same application can also be hosted as an internal website, for example on a Shiny Server or similar institutional platform. This allows educators to make the allocation interface available to course coordinators, administrators, or other staff without requiring each user to install R packages or run optimization code locally. The home tab acts as a model selector. From there, users can open one of the three allocation modules.

The diversity-based assignment (DBA) module is intended for instructors who need to form balanced tutorial, lab, project, or discussion groups from student-level attributes. Users upload the student records, select the self-formed group column, choose demographic and skill variables, tune diversity and size parameters, and export the merged group assignment. The preference-based assignment (PBA) module supports courses where self-formed groups rank topics, supervisors, project streams, or other limited slots; it follows the same pattern of uploading data, setting policy parameters, solving the model, and exporting the resulting assignment.

The multi-role workload tab validates the current-semester template and, optionally, a previous assignment output produced by `assign_job()`. It supports single-semester mode with synthetic history and exposes independent TA/GR objective weights, protected-cohort settings, role bounds, E seniority scores, solver controls, and the single-semester switch. Direct R users retain the full lower-level API, including separate TA/GR preference matrices and other extraction choices. The app builds the model through `extract_multirole_info()` and `prepare_multirole_model()`, solves it through an ROI-backed `ompr` call, and formats the multi-role output with `assign_job()`. After a successful run, the app displays solver status, objective diagnostics, a workload-distribution plot, preference attainment, and the wide assignment table.

The DBA and PBA modules download CSV outputs, while the multi-role module downloads an XLSX assignment file for further processing or record-keeping.

The frontend is a convenience layer over the same package functions rather than a separate model implementation. Solver availability still depends on installed `ROI` plugins, and `Gurobi`-specific time and iteration limits only apply when the `Gurobi` backend is selected. For deployed use, uploaded files may contain sensitive student information, so the application should be run in an institutionally

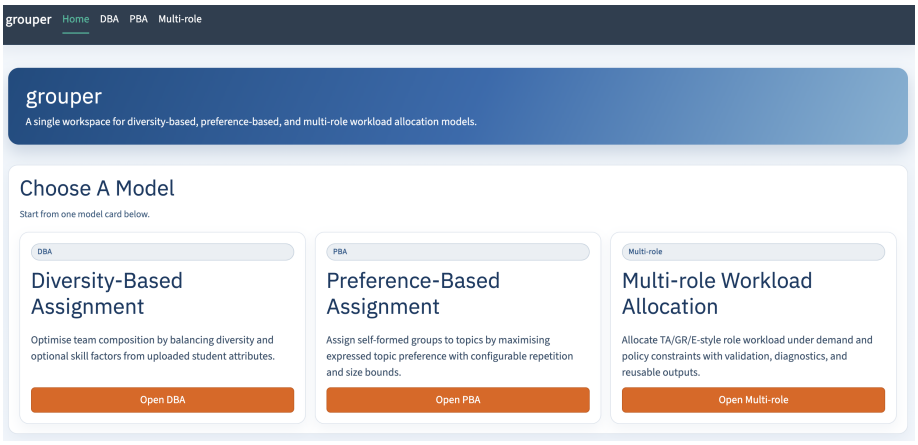

Figure 6: Home page of the web application.

Figure 7: Multi-role workload run summary and workload distribution view.

controlled environment with appropriate access restrictions.

6 Discussion

grouper is designed for higher-education allocation settings where policy rules are explicit, repeated across offerings, and difficult to manage reliably by hand. Its main advantage is that it combines three policy-aware mathematical models with a common workflow over general-purpose optimization tools. Compared with spreadsheet-based or ad hoc procedures, the package makes allocation rules explicit, records the chosen policy parameters, and returns outputs that can be mapped back to teaching and administrative records. Users nevertheless retain control over priorities such as diversity, preference satisfaction, workload balance, and protected-group constraints.

This design also creates practical limitations. Because the package encodes policy choices explicitly, it requires clean and consistently structured inputs. Richer constraints and objective weights increase flexibility, but they also increase the input burden and the need for local calibration. In practice, the objective weights should be treated as policy parameters rather than universal defaults. Institutions should confirm that the resulting trade-offs reflect their own allocation policies before using a model operationally.

Performance is governed primarily by the size and structure of the resulting mixed-integer program. The package itself is not internally parallelized. Instead, it delegates optimization to `ROI`-backed solvers through `ompr.roi`; any parallel or threaded execution depends on the selected solver and its configuration. The diversity-based model grows strongly with the number of student pairs because it uses pairwise dissimilarity terms. The preference-based model grows with the number of self-formed groups, topic-subgroup slots, and repetitions. The multi-role workload model grows with the number of individuals, tasks or courses, and role types. The three-semester examples demonstrate interactive use for moderate departmental instances, but larger deployments should be benchmarked with local data, hardware, and solver settings.

The package uses a functional interface rather than defining an S3, S4, or R6 object hierarchy. Inputs are standardized as lists, matrices, and data frames, models are represented by `ompr` objects, and

solved allocations are returned as data frames. This keeps intermediate objects inspectable, although it provides less formal encapsulation than a class-based design. The dependency structure follows the same principle of using established tools for specialized tasks. `ompr` supplies the algebraic modeling layer, while `ompr.roi` and `ROI` plugins provide access to solvers such as `GLPK`, `HiGHS`, and `Gurobi`. Packages including `cluster`, `dplyr`, and `yaml` support distance computation, data handling, and configuration parsing. This modular design avoids embedding a custom solver in **grouper**, but solver availability, installation, and licensing remain deployment considerations. Another advantage of this workflow is that it invites others to add models. To contribute to the package, which we envision to be a collection of models that are useful for educators, an investigator only has to consider pre-processing steps (for `extract_info()` to call), and a model formulation using `ompr` (for `prepare_model()` to use).

7 Conclusion

grouper provides three mathematical models and a consistent R workflow for recurring allocation problems in higher education. Diversity-based assignment supports balanced group composition, preference-based assignment allocates self-formed groups to requested destinations, and multi-role workload allocation combines demand with independently configurable role-specific preference, fairness, priority, and cohort-protection rules. Across these settings, the package translates institutional policies into explicit model terms exposed through documented R interfaces.

The detailed multi-role application demonstrates this design on a departmental planning task across three semesters. It uses a TA-focused configuration that combines prior workload, course demand, TA preferences, seniority-based priority, and Year-1 TA protection, while leaving the optional GR objective terms disabled. Under the stated policy weights, the optimized schedules obtain lower objective values than the corresponding manual schedules. This comparison is specific to the encoded policy. The generalized model supports broader TA and GR configurations, while the first two workflows extend the same data-build-solve pattern to diversity balancing and preference-sensitive topic allocation.

Overall, **grouper** fills a practical gap between low-level optimization toolkits and the allocation decisions routinely faced by instructors and administrators. Its scripted and Shiny interfaces support both reproducible analysis and operational use. Future development will focus on extending the supported allocation models, improving diagnostic summaries, and strengthening interfaces for institutional deployment while preserving the common workflow.

Bibliography

A. A. Anwar and A. S. Bahaj. Student project allocation using integer programming. *IEEE Transactions on Education*, 46(3):359–367, 2003. doi: 10.1109/TE.2003.813519. URL https://doi.org/10.1109/TE.2003.813519. [p1, 2]

Free Software Foundation. *GLPK (GNU Linear Programming Kit)*, 2026. URL https://www.gnu.org/software/glpk/. [p2]

Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*, 2026. URL https://www.gurobi.com/documentation/. [p2]

HiGHS Developers. *HiGHS: High Performance Software for Linear Optimization*, 2026. URL https://highs.dev/. [p2]

K. Hornik, S. Theussl, F. Schwendinger, M. Schwenk, and G. Stigler. *ROI: R Optimization Infrastructure*, 2024. URL https://CRAN.R-project.org/package=ROI. R package version 1.0-1. [p2]

H. Kim and C. Nolte. *golfr: Group Assignment Tool*, 2025. URL https://CRAN.R-project.org/package=golfr. R package version 0.1.0. [p2]

D. Meyer. Optassign—a web-based tool for assigning students to groups. *Computers & Education*, 53(4): 1104–1119, 2009. doi: 10.1016/j.compedu.2009.04.014. URL https://doi.org/10.1016/j.compedu.2009.04.014. [p1, 2]

J. Ramotsisi, M. Kgomotso, and L. Seboni. An optimization model for the student-to-project supervisor assignment problem: The case of an engineering department. *Journal of Optimization*, 2022(1): 9415210, 2022. doi: 10.1155/2022/9415210. URL https://doi.org/10.1155/2022/9415210. [p1]

D. Schumacher. *ompr: Model and Solve Mixed Integer Linear Programs*, 2025a. URL https://CRAN.R-project.org/package=ompr. R package version 1.0.4. [p1]

D. Schumacher. *ompr.roi: ROI Plugin for ompr*, 2025b. URL https://CRAN.R-project.org/package=ompr.roi. R package version 1.0.2. [p2]

Mingyuan Zhang
National University of Singapore
Department of Statistics and Data Science
Singapore
ORCiD: 0009-0009-1814-4962
mingyuan.z@u.nus.edu

Kevin Lam
National University of Singapore
Department of Computer Science
Singapore
ORCiD: 0000-0002-1535-7059
kevinlam@nus.edu.sg

Vik Gopal
National University of Singapore
Department of Statistics and Data Science
Singapore
ORCiD: 0000-0002-7232-2063
vik.gopal@nus.edu.sg